UNIVERSIDAD DE CALDAS — MÓDULO: PROGRAMACIÓN WEB II

PARCIAL RESUELTO: Manipulación del DOM y Gestión de Eventos

Estudiantes: Valentina Muñoz Estrada & Kevin Carmona Ospina
Docente: Andrés Felipe Osorio Quintero
Módulo: Unidad 1 - Eventos DOM
Estado: Completado / Calificado

SECCIÓN I. PREGUNTAS DE SELECCIÓN MÚLTIPLE (1 – 20)

Pregunta 1. Manejador pesado en el hilo principal
Una persona desarrolladora registra en un botón un manejador de click que recorre y procesa miles de elementos antes de terminar. Según el modelo de ejecución del navegador, ¿qué ocurre mientras el manejador se ejecuta?

A. La interfaz sigue respondiendo porque cada evento se atiende en un hilo independiente.
B. Los eventos posteriores se descartan definitivamente y nunca llegan a procesarse.
X C. No se atienden otros eventos ni se repinta la pantalla, porque el hilo principal está ocupado.
D. El navegador cancela automáticamente el manejador al detectar que tarda demasiado.

Pregunta 2. Retiro fallido de manejador con función anónima
Un equipo escribe `boton.addEventListener("click", () => guardar());` y más adelante intenta retirar ese manejador con `boton.removeEventListener("click", () => guardar());`. El manejador sigue activo. ¿Cuál es la causa más probable?

A. Las funciones flecha no admiten removeEventListener en ningún navegador.
X B. Cada función anónima es una referencia distinta, por lo que la retirada no coincide con el registro original.
C. removeEventListener solo funciona con eventos que burbujean.
D. Para retirar un manejador es obligatorio haberlo registrado con { once: true }.

Pregunta 3. Eventos de teclado e internacionalización
Se desarrolla un videojuego sencillo que debe moverse con las teclas W, A, S y D, y funcionar igual en teclados con distribución española, QWERTY o AZERTY. ¿Qué propiedad del evento de teclado resulta más adecuada y por qué?

A. evento.key, porque devuelve el carácter producido según el idioma del usuario.
B. evento.keyCode, porque es la propiedad estándar más reciente.
X C. evento.code, porque identifica la posición física de la tecla sin depender de la distribución.
D. evento.isComposing, porque informa el estado de la tecla pulsada.

Pregunta 4. Optimización de scroll con requestAnimationFrame
En un manejador de scroll se usa una bandera pendiente y se llama a requestAnimationFrame(actualizar) solo cuando la bandera es false. ¿Cuál es el efecto principal de este diseño?

A. Garantiza que el evento scroll se dispare menos veces por segundo en el navegador.
X B. Agrupa el trabajo para que se realice como máximo una actualización por repintado, descartando eventos intermedios.
C. Traslada el cálculo a un hilo secundario para liberar el hilo principal.
D. Permite llamar a preventDefault() sin perjudicar el desplazamiento.

Pregunta 5. Evento unificado para envío de formularios
Un formulario permite enviar con el botón y también pulsando Enter dentro de un campo. Para validar siempre antes de enviar, ¿qué evento conviene escuchar y por qué?

A. click del botón de envío, porque es el único que se ejecuta en todos los casos.
X B. submit del formulario, porque también se dispara al pulsar Enter y al llamar a requestSubmit().
C. change del formulario, porque se ejecuta cada vez que se modifica un campo.
D. focusout del formulario, porque se ejecuta al salir de cada control.

Pregunta 6. Restablecimiento de formularios y actualización de estado
En un formulario con un contador de caracteres restantes, la persona pulsa el botón de restablecer y el contador conserva el valor anterior. ¿Qué explica este comportamiento y cómo se corrige?

X A. reset() no dispara los eventos input ni change, por lo que el contador debe actualizarse manualmente.

B. maxlength impide que el contador se actualice tras restablecer.

C. El evento input solo funciona con teclado y no con acciones del navegador.

D. Debe reemplazarse reset() por submit() para que se disparen los eventos.

Pregunta 7. Identificación de target y currentTarget
Se registra un manejador de click en una lista <ul>. Una persona hace clic sobre un dentro de un <li> de esa lista. Dentro del manejador, ¿qué valores tienen evento.target y evento.currentTarget?

A. target es la <ul> y currentTarget es el .

B. Ambos son el , porque es el elemento donde ocurrió el clic.

C. Ambos son la <ul>, porque es el elemento que tiene el manejador.

X D. target es el y currentTarget es la <ul>.

Pregunta 8. Orden de ejecución con fase de captura
Los elementos A (abuelo), B (padre) y C (hijo) están anidados y todos tienen un manejador de click. Solo A se registró con { capture: true }; B y C usan el registro por defecto. Si se hace clic en el botón dentro de C, ¿en qué orden se ejecutan los manejadores?

X A. A, C, B

B. C, B, A

C. A, B, C

D. B, C, A

Pregunta 9. Independencia entre preventDefault() y la propagación
Un enlace <a> está dentro de un contenedor que tiene un manejador de click. El manejador del enlace llama a evento.preventDefault(). ¿Qué ocurre con el manejador del contenedor?

A. No se ejecuta, porque preventDefault() detiene la propagación.

X B. Se ejecuta normalmente, porque preventDefault() solo cancela la acción por defecto y no afecta la propagación.

C. Se ejecuta únicamente si el evento no era cancelable.

D. Se ejecuta solo en la fase de captura.

Pregunta 10. Efectos secundarios de stopPropagation()
Un componente llama a stopPropagation() para evitar que su clic active un manejador externo. Tiempo después, el cierre del menú "al hacer clic fuera" deja de funcionar para ese componente. ¿Qué principio ilustra este problema?

X A. Detener la propagación rompe el código que depende de observar el evento desde un ancestro.

B. Los manejadores de captura siempre tienen prioridad sobre los de burbujeo.

C. stopPropagation() elimina los manejadores del propio elemento.

D. Los eventos personalizados no pueden atravesar el árbol del DOM.

Pregunta 11. Manejo de elementos dinámicos en el DOM
Una aplicación ejecuta document.querySelectorAll(".eliminar").forEach(b => b.addEventListener("click", borrar)) al cargar. Los botones "Eliminar" creados después con JavaScript no responden. ¿Cuál es la causa y la solución adecuada?

X A. El selector se evalúa una sola vez; conviene delegar el evento en un ancestro estable.

B. Los botones dinámicos requieren { passive: true } para recibir eventos.

C. querySelectorAll devuelve una lista que se actualiza sola, pero forEach no la recorre de nuevo.

D. Los elementos creados con JavaScript no pueden recibir eventos de clic.

Pregunta 12. Accesibilidad en menús de navegación web
Para la navegación principal de un sitio web, se debe elegir entre aplicar roles ARIA menu y menuitem o usar un botón de divulgación con aria-expanded sobre una lista de enlaces dentro de <nav>. ¿Qué opción es la recomendada y por qué?

A. Los roles menu y menuitem, porque son los que reconocen todos los lectores de pantalla.

X B. El botón de divulgación, porque los roles menu describen menús de aplicación y exigen un modelo complejo de teclado que no corresponde a la navegación de un sitio.

C. Los roles menu, porque evitan tener que gestionar aria-expanded.

D. Ambas opciones son equivalentes; la diferencia es solo estética.

Pregunta 13. Ocultamiento accesible de componentes
Un panel desplegable se oculta únicamente con opacity: 0. Al navegar con Tab, el foco sigue pasando por sus enlaces aunque no se ven. ¿Qué solución es la más adecuada?

A. Añadir outline: none a los enlaces del panel.

X B. Usar el atributo hidden, que retira el contenido de la presentación y del árbol de accesibilidad.

C. Posicionar el panel fuera de pantalla con valores negativos.

D. Registrar un manejador de keydown en cada enlace para bloquear el foco.

Pregunta 14. Uso correcto de setCustomValidity()
En un formulario, la confirmación de contraseña usa setCustomValidity("Las contraseñas no coinciden."). Aunque la persona corrige el valor, el campo sigue marcado como inválido. ¿Cuál es el error más probable?

X A. No se restableció el mensaje personalizado con una cadena vacía cuando las contraseñas pasaron a coincidir.

B. El atributo pattern no se evalúa sobre valores no vacíos.

C. validity.valid solo se actualiza al recargar la página.

D. Debe usarse checkValidity() en lugar de setCustomValidity().

Pregunta 15. Estrategia de retroalimentación en validación
Respecto a cuándo mostrar los errores de validación, ¿cuál de las siguientes estrategias es la más aceptada según el documento?

A. Validar con cada pulsación desde el primer carácter para dar retroalimentación inmediata.

B. Validar únicamente al enviar el formulario y no antes.

X C. Validar al salir del campo y, desde entonces, revalidar mientras se escribe para que el error desaparezca al corregirlo.

D. Deshabilitar el botón de envío hasta que todos los campos sean válidos, sin mostrar mensajes.

Pregunta 16. Prevención de XSS al construir DOM dinámico
Una galería recibe títulos de imágenes desde un sistema externo y se plantea construir cada elemento con innerHTML y plantillas de cadena. ¿Qué riesgo principal introduce esta decisión y qué alternativa se recomienda por defecto con datos no controlados?

A. Un riesgo de rendimiento; se recomienda usar siempre DocumentFragment.

X B. Un riesgo de XSS si se interpolan datos no confiables; se recomienda createElement con textContent.

C. Un riesgo de incompatibilidad con <dialog>; se recomienda cloneNode.

D. Un riesgo de que se pierdan los atributos data-*; se recomienda setAttribute.

Pregunta 17. Cierre de modal al hacer clic en el backdrop
En el visor de una galería con <dialog>, el cierre al hacer clic en el fondo se implementa comprobando evento.target === visor. ¿Por qué el ejemplo coloca el contenido dentro de un contenedor interior con relleno?

A. Para que el clic sobre el contenido no burbujee hasta el documento.

B. Para que showModal() pueda trasladar el foco al diálogo.

X C. Para que el relleno quede en el contenedor interior y un clic sobre el fondo tenga como objetivo al propio <dialog>.

D. Para que ::backdrop herede los estilos del contenedor.

Pregunta 18. Alcance de los procesos ETL frente a filtros de cliente
Un panel web permite filtrar una tabla, normalizar un campo y exportar los datos a CSV. Un compañero afirma que con esto ya se implementó un proceso ETL completo. ¿Cuál es la valoración más precisa?

A. Es correcto: filtrar, transformar y exportar equivale a extraer, transformar y cargar.

X B. Es incorrecto: faltan extracción planificada desde fuentes externas, garantías de entrega y reintento, control de calidad a escala, trazabilidad y carga en un destino persistente.

C. Es correcto, siempre que el CSV se descargue en formato UTF-8.

D. Es incorrecto porque el navegador no puede transformar datos de ninguna manera.

Pregunta 19. Mitigación de inyección de fórmulas en CSV
Al exportar datos a CSV, la función escaparCsv antepone una comilla simple a los valores que comienzan con =, +, - o @. ¿Qué problema busca mitigar?

A. La corrupción del archivo por caracteres que no son UTF-8.

B. La duplicación de filas cuando el valor contiene comas.

X C. La inyección de fórmulas al abrir el archivo en una hoja de cálculo.

D. La pérdida de los ceros a la izquierda en los números.

Pregunta 20. Captura de eventos en el ciclo de vida del DOM
¿Cuál es el evento del objeto `window` que se dispara una vez que el documento HTML inicial ha sido completamente analizado y construido en el árbol DOM, sin esperar a que terminen de cargar hojas de estilo, imágenes o frames?

A. window.onload

X B. DOMContentLoaded

C. readystatechange

D. beforeunload

SECCIÓN II. PREGUNTAS DE RESPUESTA ABIERTA (21 – 25)

Pregunta 21. Modelo de ejecución del navegador y experiencia de usuario
Explique cómo funciona el modelo de ejecución del navegador (hilo principal, cola de tareas y bucle de eventos) y argumente por qué escribir manejadores breves es una cuestión de UX. Apoye su respuesta con dos consecuencias prácticas.

> **Respuesta:** El navegador opera con un Event Loop sobre un único hilo principal. Los callbacks de eventos se colocan en la Task Queue y el Event Loop los pasa al Call Stack cuando este se vacía. Entre tareas, el navegador actualiza la pantalla (60 fps). Si un manejador es pesado, bloquea el hilo principal:
> 1. **Bloqueo visual (Jank):** La UI deja de responder, las animaciones se congelan y los clics quedan represados.
> 2. **Avisos de congelamiento:** El sistema advierte que la página no responde, deteriorando la experiencia del usuario.

Pregunta 22. Comparativa: Delegación de eventos vs. Registro directo
Compare la delegación de eventos con el registro directo analizando cuatro criterios. Explique cuándo el registro directo sigue siendo la mejor opción.

> **Respuesta:**
> 1. **Elementos dinámicos:** La delegación detecta elementos creados dinámicamente sin re-registrar manejadores. El registro directo requiere adjuntar listeners manualmente a cada nuevo elemento.
> 2. **Memoria:** La delegación usa 1 solo listener en el ancestro, reduciendo el consumo. El registro directo crea N listeners.
> 3. **Eventos sin burbujeo:** Eventos como `focus`, `blur` o `mouseenter` no burbujean, dificultando la delegación.
> 4. **Sensibilidad a stopPropagation():** Si un elemento interno corta la propagación, el evento no llega al ancestro delegador.
> **¿Cuándo usar registro directo?** En componentes estáticos e independientes (ej. botones de control multimedia o canvas) que requieren interceptar eventos específicos sin depender de la jerarquía de contenedores.

Pregunta 23. Independencia de preventDefault() y stopPropagation()
Explique por qué preventDefault() y stopPropagation() son independientes, por qué evitar stopPropagation() y qué alternativas existen.

> **Respuesta:** `preventDefault()` cancela la acción nativa del navegador (navegar, enviar formulario) sin frenar el flujo del evento. `stopPropagation()` interrumpe la propagación por el árbol DOM.
> **Riesgos:** Detener la propagación crea "agujeros negros" que rompen funciones globales (analítica, menús emergentes, cierres al hacer clic fuera).
> **Alternativa:** En el manejador externo, validar si el clic ocurrió dentro de la zona de interés usando `contenedor.contains(e.target)`.

Pregunta 24. Validación en el cliente vs. Servidor y su impacto en ETL
Analice por qué la validación en el navegador no es un control de seguridad y debe repetirse en el servidor. Relaciónelo con la calidad de datos y los procesos ETL.

> **Respuesta:** La validación en cliente mejora la UX dando respuesta rápida, pero vive en un entorno manipulable por el usuario (vía DevTools o Curl). El servidor debe autenticar, sanitizar y validar todos los datos recibidos.
> **Impacto ETL:** Bajo el principio *"Garbage In, Garbage Out"*, ingresar datos inválidos o corruptos a la base de datos provoca fallos en las etapas de Extracción y Transformación, o contamina las métricas en el Data Warehouse.

Pregunta 25. Criterios de diseño para menús desplegables accesibles
Describa las decisiones de diseño para un menú accesible: aria-expanded, hidden, cierre con Escape, retorno del foco y clic fuera.

> **Respuesta:** El botón usa `aria-expanded="true|false"` para anunciar su estado a lectores de pantalla. El desplegable lleva el atributo HTML `hidden` al estar cerrado para removerlo de la navegación por teclado.
> Al presionar `Escape`, el menú se oculta y el foco regresa al botón con `boton.focus()`. Para clics externos, se escucha el `document` y se oculta si `!menu.contains(e.target) && !boton.contains(e.target)`.

SECCIÓN III. PREGUNTAS DE APLICACIÓN Y EJEMPLOS (26 – 30)

Pregunta 26. Aplicación de delegación de eventos en tabla de logística
Proponga un ejemplo de delegación para una tabla interactiva de pedidos: ancestro, evento, identificación con closest y atributos data-accion.

> **Ejemplo Práctico:** • **Anestro y Evento:** Se registra un listener de `click` en el `<tbody id="pedidos">`.
> • **Atributos Data:** Los botones usan `data-accion="aprobar"` o `data-accion="eliminar"`.
> • **Lógica JS:** `const btn = e.target.closest('[data-accion]'); if (!btn || !this.contains(btn)) return; const accion = btn.dataset.accion; const id = btn.closest('tr').dataset.id; if (accion === 'eliminar') eliminarPedido(id);`

Pregunta 27. Diseño de formulario seguro para empresa de seguros
Proponga un ejemplo con 3 campos (correo, edad, contraseña) detallando reglas HTML/API, mensajes de error y verificación de servidor.

> **Campos y Reglas:**
> 1. **Correo:** `type="email" required`. Fallo `typeMismatch`: "Ingrese un correo válido". *Servidor:* Valida estructura sintáctica y verifica registros MX.
> 2. **Edad:** `type="number" min="18" max="99" required`. Fallo `rangeUnderflow`: "Debe ser mayor de edad". *Servidor:* Corrobora contra documento de identidad.
> 3. **Contraseña:** `pattern="^(?=.*[A-Z])(?=.*\d).{8,}$"`. Fallo `patternMismatch`: "Mínimo 8 caracteres, 1 mayúscula y 1 número". *Servidor:* Aplica hash seguro (Argon2) y comprueba contraseñas filtradas.

Pregunta 28. Filtro en vivo y exportación segura a CSV en ventas
Proponga el diseño de una interfaz para filtrar y exportar datos de ventas a CSV: eventos, transformaciones ligeras y seguridad.

> **Diseño:** • **Eventos:** Escuchar `input` (con debounce de 300ms) para filtrar la tabla en vivo, y `click` para exportar CSV.
> • **Transformación:** Mapear el arreglo de ventas, formatear montos y omitir filas filtradas.
> • **Seguridad:** Usar `textContent` al insertar datos HTML para prevenir XSS. En el CSV, envolver valores entre comillas dobles y anteponer `'` si inician con `=`, `+`, `-` o `@` para evitar inyección de fórmulas en hojas de cálculo.

Pregunta 29. Analítica de clics en e-commerce y normativa colombiana
Proponga un sistema de analítica mediante delegación de eventos y envío de datos. Indique aspectos éticos y la normativa colombiana de datos.

> **Analítica y Ley 1581 de 2012:**
> • **Captura:** Delegar `click` en `document.body` para identificar botones con `data-analytics="comprar"`. Enviar métricas mediante `navigator.sendBeacon('/api/metrics', data)` sin bloquear la navegación.
> • **Normativa Colombiana:** Solicitar autorización previa e informada para el tratamiento de datos personales, anonimizar direcciones IP y garantizar los derechos Habeas Data (conocer, actualizar y suprimir información).

Pregunta 30. Estrategia de desplazamiento de alto rendimiento en portal de noticias
Proponga una estrategia para imágenes y barra de lectura: scroll + rAF, IntersectionObserver, lazy loading y prefers-reduced-motion.

> **Rendimiento y Accesibilidad:**
> • **Lectura:** Optimizar la barra de progreso vinculando el evento `scroll` con `requestAnimationFrame`.
> • **Carga de Imágenes:** Usar `loading="lazy"` nativo junto a `IntersectionObserver` para cargar imágenes solo al aproximarse al viewport.
> • **Navegación y Accesibilidad:** Preferir un botón "Cargar más" sobre el scroll infinito automático. Si la media query `prefers-reduced-motion: reduce` está activa, desactivar las animaciones suaves y aplicar cambios directos de estado.